

Report: Simple Database Management Tool Using JDBC

Course summary

The database system course aimed to apply learned knowledge and techniques to develop and design database applications for information processing. The semester covered a range of database concepts from data management evolution to advanced SQL and database design principles. The course included relational database theory, data dependencies, normalization, data storage, indexing, query optimization, and transaction management. The comprehensive study prepared students for further database studies and real-world applications.

Introduction

This report outlines the development and testing of a simple Java program designed to connect to a real database, execute SQL statements, and manage data and metadata. The tool is intended for use by Database Administrators (DBAs) as a basic management utility.

Objectives

- Connect to a database using JDBC.
- Execute SQL queries and updates.
- Retrieve and display data.
- Access and display metadata.
- Provide a text-based interface for executing SQL statements.
(including: INSERT INTO, DELETE, UPDATA...etc)

Project environment

- **MySQL:** MySQL Workbench 8.0 CE
- **JDBC:** MySQL Connector/J 9.1.0 (support MySQL8.0)
- **IDEA:** IntelliJ IDEA2020.1.2
- **JDK:** JDK14(the highest version IDEA2020 supports)

Database

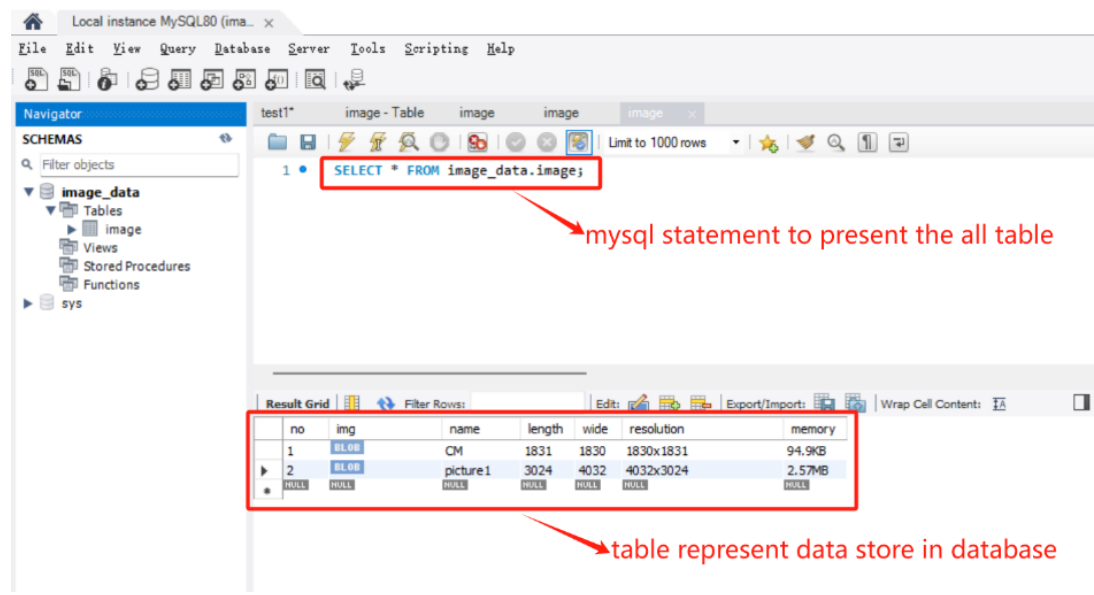


Figure1: create by MySQL: A database for managing image data.

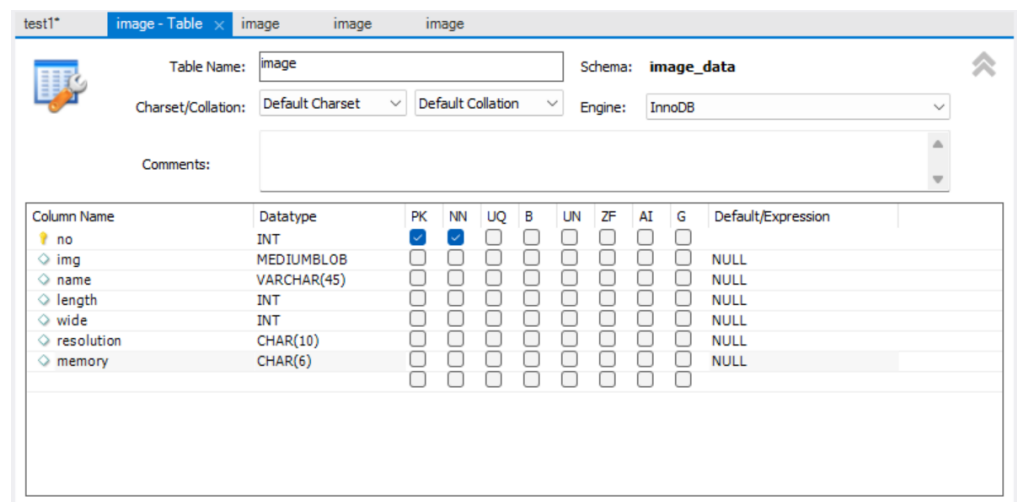


Figure2: create column_name for the table

The database architecture for this project is comprised of two interconnected tables that collectively support the function of image basic information management, its digital information.

The following ER diagram and the database table relationship diagram visually represent the structure and relationships within the database, offering a clear and concise understanding of how data is interconnected in the system.

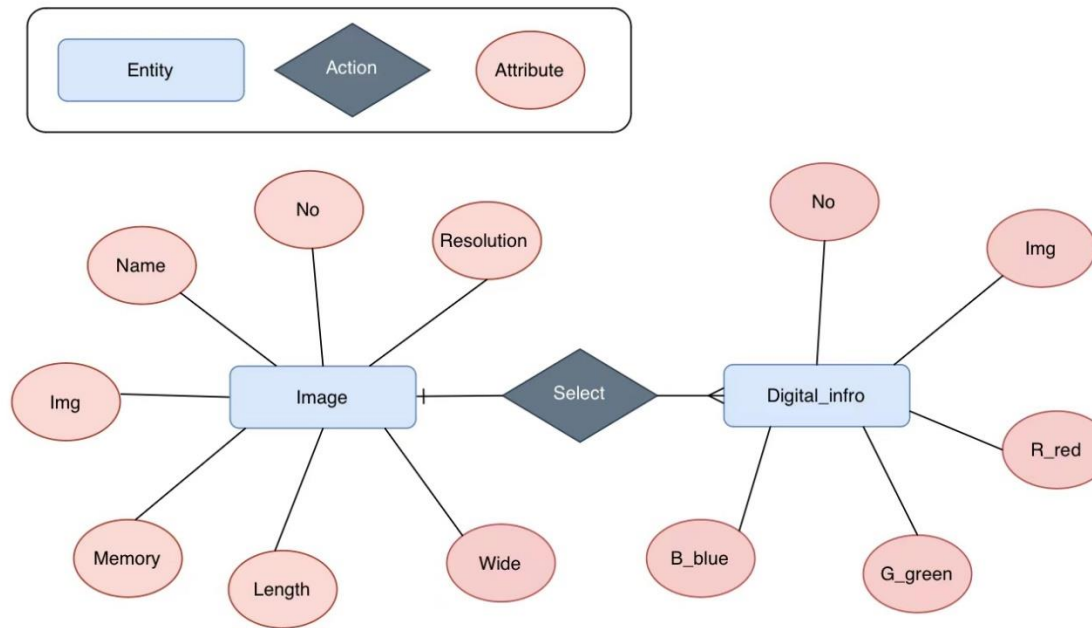


Figure 3: ER diagram of the database

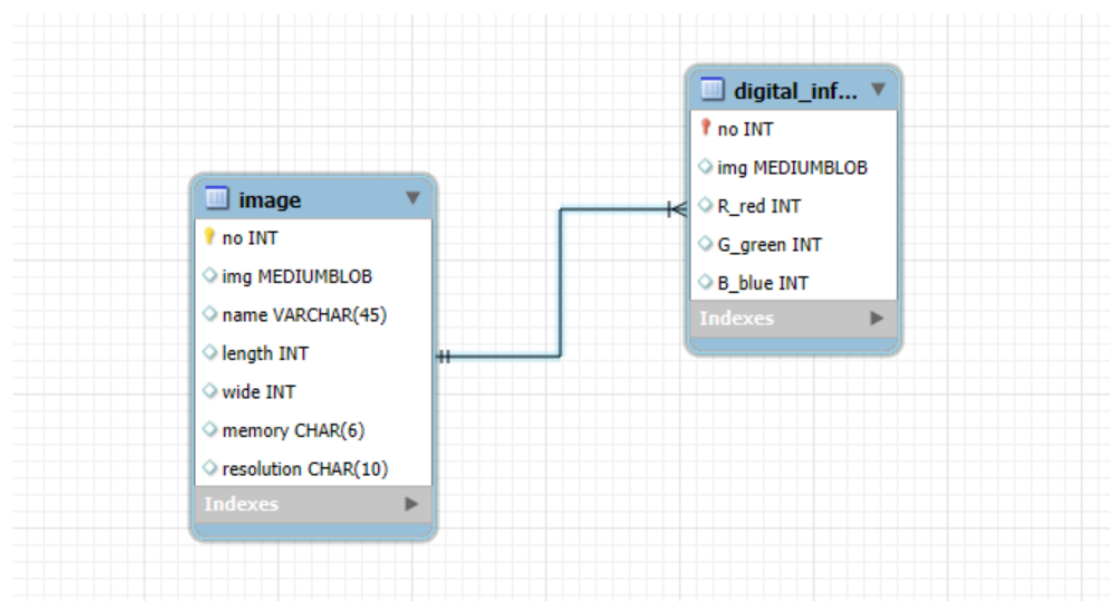


Figure 4: Crow's Foot Notation- the relationship between table ("digital_info" & "image") of my database (image_data)

Require 2 table which "digital_info" is constrained, the "number" column in the current table must have values that exist in the "number" column of the table, maintaining referential integrity.

Methodology

To manage above database, The program written was developed using Java and the JDBC API. The following steps were taken to create the tool:

Import packet

```
package com.hzj.app;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
```

1. Database Connection

- Registered the JDBC driver for the database.
- Established a connection to the database using the JDBC URL, username, and password.

```
private static final String DB_URL =
"jdbc:mysql://localhost:3306/image_data?characterEncoding=UTF-
8&useUnicode=true";
private static final String USER = "root";
private static final String PASS = "bZ8660123";
```

- **3306**: This is the default port number for the MySQL database server. If you have configured your MySQL server to listen on a different port, you would need to change this accordingly.
- **image_data**: This is the name of the database to which you are connecting. In this example, we are connecting to a database named image_data.
- **?**: The question mark signifies the end of the JDBC URL and the beginning of the parameter section.
- **characterEncoding=UTF-8**: This parameter specifies the character encoding to be used for communication with the database. UTF-8 is a widely used character encoding that supports a variety of language characters.
- **&**: The ampersand is used to separate multiple parameters.
- **useUnicode=true**: This parameter indicates that the driver should use the Unicode character set. When set to true, it is typically used in conjunction with the characterEncoding parameter to ensure proper handling of character data.

- **USER** = database user name
PASS = database password

```
// Load JDBC driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Create connection
Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
```

Connect the database by using **JDBC driver**.

**Below are the codes to perform “Data Retrieval” “Metadata Access”
“Data Update”**

```
while (true) {
    System.out.println("請輸入 SQL 語句 （輸入'exit'退出） :");
    String sql = scanner.nextLine();

    // Check if the user wants to exit
    if (sql.equalsIgnoreCase("exit")) {
        break;
    }

    // Determine the type of SQL statement
    if (sql.trim().toUpperCase().startsWith("SELECT")) {
        executeQuery(conn, sql);
    } else {
        executeUpdate(conn, sql);
    }
}
```

To performs the search and update operation

2. Data Retrieval

- Created a Statement object to execute a SQL query.
- Executed a SELECT query to retrieve data from a specified table.
- Iterated over the ResultSet to display the data in a text format.

3. Metadata Access

- Used the `ResultSetMetaData` object to retrieve metadata about the columns in the result set.
- Displayed the column names and types.

```
private static void executeQuery(Connection conn, String sql) throws
SQLException {
    try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        // Display column names with proper spacing
        for (int i = 1; i <= rs.getMetaData().getColumnCount(); i++) {
            String columnName = rs.getMetaData().getColumnName(i);
            System.out.printf("%-" + COLUMN_WIDTH + "s", columnName); // Left-
aligned with padding
        }
        System.out.println();

        // Display query results with proper formatting
        while (rs.next()) {
            for (int i = 1; i <= rs.getMetaData().getColumnCount(); i++) {
                Object value = rs.getObject(i);
                String valueString = (value != null) ? value.toString() : "NULL";

                // Truncate if the value is too long
                if (valueString.length() > MAX_VALUE_LENGTH) {
                    valueString = valueString.substring(0, MAX_VALUE_LENGTH - 3)
+ "...";
                }

                System.out.printf("%-" + COLUMN_WIDTH + "s", valueString); // Left-
aligned with padding
            }
            System.out.println();
        }
    }
}
```

Display the selected attribute with proper spacing. In order, the data have different type like 'int', 'blob', 'double'... etc, we use '`rs.getObject(i)`' rather than '`rs.getString(i)`'.

4. Data Update

- Executed INSERT INTO, DELETE, UPDATE...etc SQL statement to modify data in the database.
- Displayed the number of rows affected by the update.

```
private static void executeUpdate(Connection conn, String sql) throws
SQLException {
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        int affectedRows = pstmt.executeUpdate();
        System.out.println("操作成功， 影響的行數: " + affectedRows);
    }
}
```

This line starts a private static method named `executeUpdate` that takes a `Connection` object and an SQL string as parameters and throws an SQL Exception. The `executeUpdate` method contains code to execute SQL statements other than `SELECT` (like `INSERT`, `UPDATE`, `DELETE`), and it prints the number of affected

5. Cleanup

- Closed the `ResultSet`, `Statement`, and `Connection` objects to free up resources.

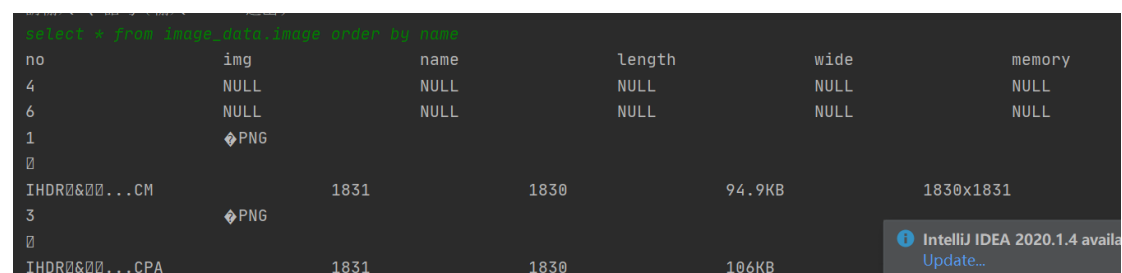
```
if (sql.equalsIgnoreCase("exit")) {
    break;
}
```

Observation

The program successfully connected to the database and performed the following operations:

Data Retrieval, Metadata Access

The `SELECT` query executed without errors, and the following data was retrieved:



no	img	name	length	wide	memory
4	NULL	NULL	NULL	NULL	NULL
6	NULL	NULL	NULL	NULL	NULL
1					
0					
IHDR	...	CM	1831	1830	94.9KB
3					
0					
IHDR	...	CPA	1831	1830	106KB

Before improve version1 (using `rs.getString(i)`)

```
"C:\Program Files\Java\jdk-14.0.1\bin\java.exe" "-javaagent:D:\develop_app\Int
請輸入SQL語句（輸入'exit'退出）：
SELECT * FROM image_data.image;
no  img name      length  wide    memory  resolution
1   [B@38e79ae3 CM    1831    1830    94.9KB  1830x1831
2   [B@63070bab picture1 3024    4032    2.57MB  4032x3024
3   [B@68e5eea7 CPA    1831    1830    123     1831x1830
4   null      null     null     null     null     null
6   null      null     null     null     null     null
請輸入SQL語句（輸入'exit'退出）：
```

Before improve version2 (using rs.getObject(i))

```
"C:\Program Files\Java\jdk-14.0.1\bin\java.exe" "-javaagent:D:\develop_app\IntelliJidea\IntelliJ IDEA 2020.1.2\lib\idea_rt.jar=49863:D:\
請輸入SQL語句（輸入'exit'退出）：
select * from image_data.image order by name desc;
no      img      name      length      wide      memory      resolution
2       [B@58ea606c picture1    3024        4032        2.57MB      4032x3024
3       [B@6f45df59 CPA         1831        1830        106KB       1831x1830
1       [B@38e79ae3 CM          1831        1830        94.9KB      1830x1831
4       NULL      NULL       NULL        NULL        NULL        NULL
6       NULL      NULL       NULL        NULL        NULL        NULL
請輸入SQL語句（輸入'exit'退出）：
```

After improve Table of 'image' table(with proper spacing)

When we type 'select * from image_data.image order by name', table is provided to view.

Data Update

The UPDATE statement executed successfully, and the following message was displayed:

INSERT INTO

```
"C:\Program Files\Java\jdk-14.0.1\bin\java.exe" "-javaage
請輸入SQL語句（輸入'exit'退出）：
INSERT INTO image_data.image(no) VALUES(3);
操作成功，影響的行數：1
請輸入SQL語句（輸入'exit'退出）：
```

Result Grid							
Filter Rows:							
	no	img	name	length	wide	resolution	memory
	1	[B@0E	CM	1831	1830	1830x1831	94.9KB
	2	[B@0E	picture1	3024	4032	4032x3024	2.57MB
	3	NULL	NULL	NULL	NULL	NULL	NULL
	4	NULL	NULL	NULL	NULL	NULL	NULL

We reflash the data in MySQL, "3" is inserted

DELETE

```
"C:\Program Files\Java\jdk-14.0.1\bin\java.exe" "-javaagent:D:\develop_app\IntelliJ"
請輸入SQL語句（輸入'exit'退出）：
INSERT INTO image_data.image(no) VALUES(5);
操作成功，影響的行數： 1
請輸入SQL語句（輸入'exit'退出）：
DELETE FROM image_data.`image` WHERE (`no` = '5');
操作成功，影響的行數： 1
請輸入SQL語句（輸入'exit'退出）：
```

Result Grid							
Filter Rows:							
	no	img	name	length	wide	resolution	memory
▶	1	BLOB					94.9KB
	2	BLOB	picture1	3024	4032	4032x3024	2.57MB
	3	BLOB	CPA	1831	1830	1831x1830	123
	4	NULL	NULL	NULL	NULL	NULL	NULL
	5	NULL	NULL	NULL	NULL	NULL	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Before delete

Result Grid							
Filter Rows:							
	no	img	name	length	wide	memory	resolution
	1	BLOB	CM	1831	1830	94.9KB	1830x1831
	2	BLOB	picture1	3024	4032	2.57MB	4032x3024
	3	BLOB	CPA	1831	1830	106KB	1831x1830
▶	4	NULL	NULL	NULL	NULL	NULL	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

After delete

Update

```
"C:\Program Files\Java\jdk-14.0.1\bin\java.exe" "-javaagent:D:\develop_app\IntelliJ\Idea\IntelliJ IDEA 2020.1.2\lib\idea_rt.jar=57"
請輸入SQL語句（輸入'exit'退出）：
UPDATE image_data.`image` SET `img` = LOAD_FILE('D:\image\1\data\uploads\project_img\origin\CPA.png') WHERE (`no` = '3');
操作成功，影響的行數： 1
請輸入SQL語句（輸入'exit'退出）：
INSERT INTO image_data.image(no) VALUES(4);
操作成功，影響的行數： 1
請輸入SQL語句（輸入'exit'退出）：
```

Result Grid							
Filter Rows:							
	no	img	name	length	wide	resolution	memory
▶	1	BLOB	CM	1831	1830	1830x1831	94.9KB
	2	BLOB	picture1	3024	4032	4032x3024	2.57MB
	3	NULL	CPA	1831	1830	1831x1830	123
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Before update

Result Grid		Filter Rows:		Edit:		Export/Import:	
	no	img	name	length	wide	resolution	memory
▶	1	BLOB	CM	1831	1830	1830x1831	94.9KB
	2	BLOB	picture1	3024	4032	4032x3024	2.57MB
	3	BLOB	CPA	1831	1830	1831x1830	123
	4	NULL	NULL	NULL	NULL	NULL	NULL
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

After update and insert

Conclusion

The simple database management tool met the objectives of connecting to a database, executing SQL statements, and managing data and metadata. The tool provides a basic text-based interface for DBAs to perform simple operations on a database.

Improvement

- Implement error handling to manage exceptions and provide user feedback.
- Enhance security by using prepared statements to prevent SQL injection.
- Add functionality to handle more complex SQL statements and database operations.
- Develop a user-friendly interface for easier interaction (GUI) with the tool.